COMPLETE SECURITY & PRODUCTION READINESS PROMPT LIBRARY

Copy-ready defensive prompts for AI app builders, full-stack projects, APIs, SaaS products, and production deployments.

Includes: authentication • authorization • uploads • abuse prevention • caching • backend resilience • load balancing • secrets • AI crawler readiness

How to use this guide

Choose the prompt matching the problem, paste it into your coding assistant, and attach the relevant repository context. Ask the assistant to inspect first, implement in small changes, run tests, and report what it could not verify. These prompts are defensive engineering specifications—not a substitute for a professional security review.

Important: Never paste live API keys, passwords, private customer data, production tokens, or payment-card data into an AI chat. Use placeholders and your platform's secret manager.

Prepared from the security and production-readiness topics visible in the supplied screenshots. Where screenshot text was too small or obscured, this guide expands the identifiable topic into a complete, safer implementation brief.

Contents

A practical index of the problem areas covered.

1. Authentication, sessions, email verification and 2FA
2. Authorization, RBAC, tenant isolation and database RLS
3. Rate limiting, brute-force protection and abuse controls
4. Free-trial abuse and privacy-respecting risk scoring
5. Secure file upload and storage pipeline
6. Input validation, error handling and safe output
7. Secrets, environment configuration and credential rotation
8. Dependency and software supply-chain security
9. Cache stampede, TTL jitter and request coalescing
10. Cache poisoning, invalidation and tenant-safe caching
11. Backend resilience, timeouts, retries and circuit breakers
12. Database performance, pooling and query safety
13. Load balancing, health checks and deployment topology
14. DDoS readiness, edge controls and traffic degradation
15. Frontend security, XSS, CSRF and browser hardening
16. API security, IDOR and sensitive data exposure
17. Logging, monitoring, alerting and incident response
18. Production QA and security acceptance checklist
19. GEO / LLM discovery files and crawler validation
20. Master all-in-one security audit and remediation prompt

1. Authentication, sessions, email verification and 2FA

Addresses the screenshot’s authentication hardening requirements: secure cookies, server-side session checks, verified-email gates, stronger account protection, rate limits, and temporary lockouts.

COPY-READY BUILDER PROMPT

```
ROLE
Act as a senior application security engineer, backend architect, and pragmatic code reviewer.

OBJECTIVE
Harden sign-in, sign-up, password reset, email verification, session renewal, logout, and optional multi-factor
authentication. Prevent session theft, account enumeration, credential stuffing, and privilege escalation.

PROJECT CONTEXT
Inspect the existing repository and adapt to its actual framework, database, hosting provider, and conventions. Do
not replace working architecture without evidence.

REQUIRED WORK
1. Inventory the current auth provider and every route, server action, middleware function, and database table
involved in authentication.
2. Store session credentials only in appropriately configured Secure, HttpOnly cookies; select SameSite=Lax or Strict
based on legitimate cross-site flows. Never put session tokens in localStorage, sessionStorage, URLs, or
browser-readable persistent storage.
3. Enforce authentication and authorization on the server for every protected operation. Re-check current user
identity, account status, role, and resource ownership at the point of access.
4. Require verified email before protected account actions. Make verification tokens single-use, time-limited, high
entropy, and stored safely as hashes where supported.
5. Add optional TOTP or passkey-based MFA for sensitive accounts and actions. Provide recovery codes that are
one-time, hashed at rest, and shown only once.
6. Protect password reset and email-change flows with expiring single-use tokens, notification of security changes,
and session revocation where appropriate.
7. Implement login, signup, reset, verification, and MFA throttling with per-account and per-network controls,
progressive delays, generic public errors, and a safe recovery path.
8. Rotate session identifiers after login, MFA completion, and privilege changes. Revoke sessions on logout, password
reset, account compromise, and administrative revocation.
9. Use a maintained password hashing algorithm through the auth provider; do not invent a custom password-hashing
scheme.

SECURITY AND QUALITY RULES
- Do not implement permanent lockouts that let attackers deny service to a victim; use bounded temporary throttles
and safe recovery.
- Do not reveal whether an email address exists through public error messages or materially different response
behavior.
- Do not log passwords, verification links, recovery codes, session cookies, or authorization headers.
- Use provider-supported secure cookie settings and document any cross-domain constraints.

TESTS AND ACCEPTANCE
- A logged-out user cannot access protected APIs by calling them directly.
- A user cannot access another user's account or change their role through client-side requests.
- Unverified accounts are blocked from the protected actions specified by product policy.
- Cookie flags, session rotation, expiry, revocation, MFA, and recovery behavior are tested.
- Rate limits work across multiple app instances and return safe retry guidance.

IMPLEMENTATION EXPECTATION
First inspect the codebase and explain the relevant current flow briefly. Then implement the changes, not merely
describe them. If a required secret or provider setting is missing, add a clearly named environment-variable
placeholder and document how an authorized maintainer configures it. Never invent credentials.
```

Implementation note: use the authentication provider’s supported session and MFA mechanisms rather than rolling your own token system.

2. Authorization, RBAC, tenant isolation and database RLS

Prevents insecure direct object references (IDOR), broken access control, cross-tenant data leakage, and client-only permission checks.

COPY-READY BUILDER PROMPT

```
ROLE
Act as a senior application security engineer, backend architect, and pragmatic code reviewer.

OBJECTIVE
Ensure every user, organization, team, admin, and service role can access only the resources and actions explicitly
granted to it.

PROJECT CONTEXT
Inspect the existing repository and adapt to its actual framework, database, hosting provider, and conventions. Do
not replace working architecture without evidence.

REQUIRED WORK
1. Map roles and permissions into a clear server-side policy model (for example owner, admin, member, read-only,
support). Avoid scattered role string checks.
2. For every read, update, delete, export, upload, and background job, verify both the caller’s permission and the
resource’s ownership or tenant membership.
3. Use database Row Level Security (RLS) where supported. Enable RLS on all user- or tenant-owned tables and write
explicit SELECT, INSERT, UPDATE, and DELETE policies.
4. Ensure service-role or admin credentials are used only in trusted server environments and never exposed to browser
bundles.
5. Derive user and tenant identifiers from the authenticated server session, not from request body fields supplied by
the client.
6. Prevent mass assignment: allowlist writable fields and reject attempts to modify role, owner_id, tenant_id,
billing status, or other privileged fields.
7. Check authorization again inside background jobs and scheduled tasks using a trusted job identity and explicit
scope.
8. Create a resource access matrix and document exceptions such as public resources or support access.

SECURITY AND QUALITY RULES
- Default to deny when identity, tenant, ownership, or permission is missing or ambiguous.
- Never trust hidden UI controls as an authorization mechanism.
- Do not bypass RLS for convenience; document and tightly scope any privileged server operation.
- Avoid leaking whether another tenant’s resource exists.

TESTS AND ACCEPTANCE
- Automated tests cover owner, same-tenant member, other-tenant member, anonymous user, and admin cases.
- Changing IDs in URL, query, or request body never grants access to another user's resource.
- Direct database access through public client credentials remains constrained by RLS.
- Privileged fields cannot be modified through ordinary user endpoints.

IMPLEMENTATION EXPECTATION
First inspect the codebase and explain the relevant current flow briefly. Then implement the changes, not merely
describe them. If a required secret or provider setting is missing, add a clearly named environment-variable
placeholder and document how an authorized maintainer configures it. Never invent credentials.
```

3. Rate limiting, brute-force protection and abuse controls

COPY-READY BUILDER PROMPT

```
ROLE
Act as a senior application security engineer, backend architect, and pragmatic code reviewer.

OBJECTIVE
Protect authentication, search, contact forms, uploads, expensive AI endpoints, payment-related actions, and public APIs from automated abuse while preserving legitimate use.

PROJECT CONTEXT
Inspect the existing repository and adapt to its actual framework, database, hosting provider, and conventions. Do not replace working architecture without evidence.

REQUIRED WORK
1. Inventory endpoints and classify them by cost and abuse risk. Define configurable limits by authenticated user, API key, IP/network, route, and tenant as appropriate.
2. Use a shared store such as Redis or a managed equivalent so limits remain consistent across horizontally scaled instances. Use atomic operations or a vetted rate-limit library.
3. Apply stricter policies to login, password reset, verification email, OTP, signup, and invitation endpoints. Add progressive delays and bounded temporary throttles.
4. Return HTTP 429 with a consistent error body and Retry-After where meaningful. Do not disclose internal risk scores or detection rules.
5. Use request-size limits, concurrency limits, queue bounds, and per-tenant quotas for expensive operations.
6. Add idempotency keys to payment and other non-repeatable operations where appropriate.
7. Trust forwarded client IP headers only when requests arrive through explicitly trusted proxies; otherwise ignore client-supplied forwarding headers.
8. Provide safe allowlisting for internal health checks and trusted jobs, with credentials or network controls—not a public bypass parameter.
9. Expose metrics for blocked requests, throttled routes, latency, and quota consumption without logging sensitive payloads.

SECURITY AND QUALITY RULES
- Do not rely on in-memory counters in a multi-instance deployment.
- Avoid a single global limit that lets one customer exhaust capacity for everyone.
- Do not block users solely because they use a VPN, shared network, mobile carrier NAT, or privacy feature.
- Fail safely during rate-limit-store outages: define stricter local emergency limits for high-risk routes and document the behavior.

TESTS AND ACCEPTANCE
- Parallel requests cannot exceed configured limits due to race conditions.
- Limits work across multiple application instances.
- 429 responses include consistent retry guidance.
- Tests cover anonymous, authenticated, tenant-level, burst, sustained, and store-outage cases.

IMPLEMENTATION EXPECTATION
First inspect the codebase and explain the relevant current flow briefly. Then implement the changes, not merely describe them. If a required secret or provider setting is missing, add a clearly named environment-variable placeholder and document how an authorized maintainer configures it. Never invent credentials.
```

4. Free-trial abuse and privacy-respecting risk scoring

The screenshots describe device, network, email and payment signals. This version avoids invasive fingerprinting and avoids treating any single signal as proof of fraud.

COPY-READY BUILDER PROMPT

```
ROLE
Act as a senior application security engineer, backend architect, and pragmatic code reviewer.

OBJECTIVE
Reduce repeated trial claims while minimizing false positives, respecting privacy, and giving legitimate users a fair appeal or verification path.

PROJECT CONTEXT
Inspect the existing repository and adapt to its actual framework, database, hosting provider, and conventions. Do not replace working architecture without evidence.

REQUIRED WORK
1. Define the trial policy clearly: eligible plans, trial duration, account limits, household/team rules, and exceptions.
2. Use first-party account history, verified email, prior trial entitlement, organization membership, and payment-provider customer references as primary signals.
3. Where lawful and disclosed, use coarse network reputation or short-lived device-risk signals only as supplementary indicators. Avoid covert or persistent invasive browser fingerprinting.
4. Never store raw card numbers or CVV. Use payment-provider token/customer identifiers and let the provider handle sensitive payment data.
5. Create a transparent risk score with documented factors, bounded weights, expiry periods, and reason codes. Do not use protected or sensitive personal attributes.
6. Use graduated decisions: allow trial, request additional verification, route to manual review, or deny only when strong policy evidence exists.
7. Do not automatically deny based only on VPN/Tor use, IP address, device similarity, or shared public networks.
8. Store minimal audit evidence with retention limits and provide a support appeal path for false positives.
9. Make trial entitlement checks atomic and idempotent so concurrent signup requests cannot claim multiple trials.

SECURITY AND QUALITY RULES
- Follow applicable privacy, consumer-protection, and payment rules; provide notice where required.
- Do not attempt to identify a person through covert cross-site tracking or browser fingerprinting.
- Keep risk features explainable, proportionate, and periodically reviewed for false positives.
- Never store payment-card details directly unless the business is properly certified and has a compelling need.

TESTS AND ACCEPTANCE
- Concurrent requests cannot create duplicate trial entitlements.
- Risk decisions are logged with non-sensitive reason codes and can be reviewed.
- Legitimate users have a verification and appeal route.
- Tests cover shared IPs, VPN use, repeat accounts, payment-provider references, and false-positive scenarios.

IMPLEMENTATION EXPECTATION
First inspect the codebase and explain the relevant current flow briefly. Then implement the changes, not merely describe them. If a required secret or provider setting is missing, add a clearly named environment-variable placeholder and document how an authorized maintainer configures it. Never invent credentials.
```

5. Secure file upload and storage pipeline

Covers the screenshot’s secure upload pipeline: MIME/type checks, file signatures, safe filenames, isolated storage, size limits, and non-executable delivery.

COPY-READY BUILDER PROMPT

```
ROLE
Act as a senior application security engineer, backend architect, and pragmatic code reviewer.

OBJECTIVE
Prevent malicious uploads, content-type spoofing, path traversal, accidental public exposure, storage exhaustion, and execution of uploaded content.

PROJECT CONTEXT
Inspect the existing repository and adapt to its actual framework, database, hosting provider, and conventions. Do not replace working architecture without evidence.

REQUIRED WORK
1. Define an explicit allowlist of extensions, detected MIME types, and file-signature rules for each upload use case. Do not trust the browser-provided Content-Type or filename.
2. Enforce per-file, per-user, per-tenant, and total storage limits at the server and storage-provider layers. Reject oversized requests before buffering them fully in memory.
3. Generate a random server-side object key. Treat the original filename as untrusted display metadata; normalize it and never use it as a filesystem path.
4. Store uploads in a private bucket or outside the web root. Do not execute or render untrusted uploads from the application origin.
5. Use short-lived, narrowly scoped signed upload/download URLs where appropriate. Verify ownership and authorization before issuing them.
6. Scan uploads with a maintained malware-scanning service where appropriate; quarantine files until scanning and validation complete.
7. For images, decode and re-encode supported formats when practical, strip unnecessary metadata, and limit dimensions and decompression expansion.
8. Protect archive handling against path traversal, zip bombs, nested archive abuse, and excessive file counts; reject archives if not required.
9. Set safe download headers such as Content-Disposition: attachment and X-Content-Type-Options: nosniff. Serve user content from a separate origin when possible.
10. Record upload status, owner, detected type, size, checksum, scan status, and retention metadata without logging file contents.

SECURITY AND QUALITY RULES
- Never accept executable formats unless the product has a specific, reviewed requirement.
- Never construct storage paths from untrusted filenames.
- Do not make uploads public by default.
- Do not trust file extensions, MIME headers, or client-side validation alone.

TESTS AND ACCEPTANCE
- Spoofed MIME types and mismatched file signatures are rejected.
- Oversized, malformed, and unauthorized uploads are rejected safely.
- A user cannot retrieve another user's private upload by changing an object key.
- Uploaded HTML or script content cannot execute in the app’s trusted origin.
- Tests cover path traversal strings, archive edge cases, and storage permission boundaries.

IMPLEMENTATION EXPECTATION
First inspect the codebase and explain the relevant current flow briefly. Then implement the changes, not merely describe them. If a required secret or provider setting is missing, add a clearly named environment-variable placeholder and document how an authorized maintainer configures it. Never invent credentials.
```

6. Input validation, error handling and safe output

COPY-READY BUILDER PROMPT

```
ROLE
Act as a senior application security engineer, backend architect, and pragmatic code reviewer.

OBJECTIVE
Prevent injection, malformed data, mass assignment, unsafe rendering, and accidental disclosure of internal errors.

PROJECT CONTEXT
Inspect the existing repository and adapt to its actual framework, database, hosting provider, and conventions. Do
not replace working architecture without evidence.

REQUIRED WORK
1. Inventory all API routes, server actions, webhooks, forms, query parameters, headers, and background-job payloads.
2. Create shared schemas with a maintained validation library (for example Zod in TypeScript). Validate type, length,
range, format, enum values, and nested object shape on the server.
3. Use parameterized database queries or the ORM's safe query APIs. Never concatenate untrusted input into SQL or
shell commands.
4. Use context-appropriate output encoding. Avoid unsafe HTML insertion; sanitize rich text with a maintained
allowlist sanitizer if HTML is a product requirement.
5. Apply CSRF protections to cookie-authenticated state-changing requests, with origin checks and framework-supported
CSRF defenses.
6. Return stable public error codes and safe messages. Keep stack traces and detailed diagnostics only in protected
server logs.
7. Validate webhook signatures, timestamps, and replay protection before processing external events.
8. Set request body, upload, pagination, and query-complexity limits.

SECURITY AND QUALITY RULES
- Treat the server as the trust boundary; never rely on client-side checks for authorization.
- Validate all untrusted input on the server using schemas and allowlists.
- Use least privilege, safe defaults, clear errors, and structured audit logging.
- Do not expose secrets, tokens, personal data, stack traces, or internal identifiers in client responses or logs.
- Preserve existing features and write focused, maintainable changes.

TESTS AND ACCEPTANCE
- Invalid input returns a predictable 400/422 response without leaking internals.
- Stored and rendered user content cannot execute scripts.
- SQL and command execution paths use safe parameterization or fixed allowlisted commands.
- Webhook signature failures and replay attempts are rejected.

IMPLEMENTATION EXPECTATION
First inspect the codebase and explain the relevant current flow briefly. Then implement the changes, not merely
describe them. If a required secret or provider setting is missing, add a clearly named environment-variable
placeholder and document how an authorized maintainer configures it. Never invent credentials.
```

7. Secrets, environment configuration and credential rotation

COPY-READY BUILDER PROMPT

```
ROLE
Act as a senior application security engineer, backend architect, and pragmatic code reviewer.

OBJECTIVE
Prevent API keys, database credentials, signing keys, and service-role tokens from entering source control, browser
bundles, logs, build artifacts, or public error messages.

PROJECT CONTEXT
Inspect the existing repository and adapt to its actual framework, database, hosting provider, and conventions. Do
not replace working architecture without evidence.

REQUIRED WORK
1. Scan tracked files, configuration, CI logs where accessible, and build output for likely secrets. Report locations
without printing full secret values.
2. Move secrets to the hosting platform’s secret manager or protected environment variables. Separate public
build-time variables from server-only secrets.
3. Ensure privileged keys are never imported into client-side code and are not prefixed with framework-specific
public variable markers.
4. Add safe .env.example placeholders, update ignore rules, and document required variables and environments.
5. If a real secret is found in repository history or logs, treat it as compromised: recommend revocation and
rotation with the provider; removing it from the latest commit is not enough.
6. Use separate credentials for development, staging, and production, with least-privilege scopes and rotation
procedures.
7. Avoid printing environment variables or secret values in diagnostics.

SECURITY AND QUALITY RULES
- Treat the server as the trust boundary; never rely on client-side checks for authorization.
- Validate all untrusted input on the server using schemas and allowlists.
- Use least privilege, safe defaults, clear errors, and structured audit logging.
- Do not expose secrets, tokens, personal data, stack traces, or internal identifiers in client responses or logs.
- Preserve existing features and write focused, maintainable changes.

TESTS AND ACCEPTANCE
- A production build contains no server-only credentials in client assets.
- The repository contains placeholders rather than live secrets.
- Secret rotation steps and ownership are documented.
- CI checks detect likely accidental secret commits.

IMPLEMENTATION EXPECTATION
First inspect the codebase and explain the relevant current flow briefly. Then implement the changes, not merely
describe them. If a required secret or provider setting is missing, add a clearly named environment-variable
placeholder and document how an authorized maintainer configures it. Never invent credentials.
```

8. Dependency and software supply-chain security

COPY-READY BUILDER PROMPT

```
ROLE
Act as a senior application security engineer, backend architect, and pragmatic code reviewer.

OBJECTIVE
Reduce risk from vulnerable, abandoned, compromised, or unexpectedly changed packages and build tools.

PROJECT CONTEXT
Inspect the existing repository and adapt to its actual framework, database, hosting provider, and conventions. Do
not replace working architecture without evidence.

REQUIRED WORK
1. Identify package managers, lockfiles, direct and transitive dependencies, runtime versions, build scripts, and
deployment workflows.
2. Run supported dependency and secret scans. Report package name, installed version, advisory identifier, severity,
reachability where known, and recommended safe version.
3. Prioritize reachable critical/high vulnerabilities and upgrade to compatible patched versions. Avoid blind
major-version upgrades.
4. Commit and enforce lockfiles; use reproducible installs in CI and avoid floating dependency versions in
production.
5. Review install and postinstall scripts for unexpected network access or file modification.
6. Pin CI actions and external build images to immutable versions or digests where supported.
7. Use least-privilege CI tokens, protected deployment environments, and required review for production releases.
8. Generate or maintain a software bill of materials if supported by the stack.

SECURITY AND QUALITY RULES
- Treat the server as the trust boundary; never rely on client-side checks for authorization.
- Validate all untrusted input on the server using schemas and allowlists.
- Use least privilege, safe defaults, clear errors, and structured audit logging.
- Do not expose secrets, tokens, personal data, stack traces, or internal identifiers in client responses or logs.
- Preserve existing features and write focused, maintainable changes.

TESTS AND ACCEPTANCE
- Dependency audit results are recorded with remediation status.
- Lockfile-based clean installation succeeds in CI.
- No known unaddressed critical vulnerability remains without an explicit risk exception.
- CI credentials have minimum required permissions.

IMPLEMENTATION EXPECTATION
First inspect the codebase and explain the relevant current flow briefly. Then implement the changes, not merely
describe them. If a required secret or provider setting is missing, add a clearly named environment-variable
placeholder and document how an authorized maintainer configures it. Never invent credentials.
```

9. Cache stampede, TTL jitter and request coalescing

Addresses the screenshot’s cache stampede, Redis Bloom filter, TTL jitter, and single-flight request coalescing topics.

COPY-READY BUILDER PROMPT

```
ROLE
Act as a senior application security engineer, backend architect, and pragmatic code reviewer.

OBJECTIVE
Reduce database and API load during cache expiry, bursts, and concurrent requests while keeping data correctness.

PROJECT CONTEXT
Inspect the existing repository and adapt to its actual framework, database, hosting provider, and conventions. Do
not replace working architecture without evidence.

REQUIRED WORK
1. Map expensive read paths and define cache keys, value schemas, freshness requirements, and invalidation events.
2. Add bounded TTL jitter so many keys do not expire simultaneously. Keep the jitter range configurable and ensure it
never exceeds the data freshness policy.
3. Implement request coalescing (single-flight) so concurrent misses for the same key share one in-flight
computation. Ensure waiters receive the same result or safe error.
4. Use short-lived negative caching only for appropriate not-found results; never cache authorization failures or
sensitive error responses.
5. Consider stale-while-revalidate for data where brief staleness is acceptable. Define maximum stale age and refresh
behavior.
6. Use Redis or a shared cache for multi-instance deployments. Add connection timeouts, bounded retries, and
circuit-breaker behavior.
7. Use Redis Bloom filters only when membership pre-checks are valuable and false positives are acceptable; always
confirm positive results against the source of truth.
8. Add cache metrics for hit rate, miss rate, evictions, refresh duration, lock contention, and backend load.

SECURITY AND QUALITY RULES
- Cache keys must include all relevant tenant, locale, permission, and query dimensions.
- Never let one user receive another user’s private or permission-dependent cached response.
- Do not cache one-time tokens, password reset responses, payment results, or other secrets.
- Locks must have bounded expiry and safe recovery if the lock holder crashes.

TESTS AND ACCEPTANCE
- A burst of concurrent misses triggers one origin computation per key within the coalescing window.
- TTL jitter spreads expirations without violating freshness requirements.
- Cache outage does not cause unbounded retries or memory growth.
- Tests verify tenant separation and invalidation after writes.

IMPLEMENTATION EXPECTATION
First inspect the codebase and explain the relevant current flow briefly. Then implement the changes, not merely
describe them. If a required secret or provider setting is missing, add a clearly named environment-variable
placeholder and document how an authorized maintainer configures it. Never invent credentials.
```

10. Cache poisoning, invalidation and tenant-safe caching

COPY-READY BUILDER PROMPT

```
ROLE
Act as a senior application security engineer, backend architect, and pragmatic code reviewer.

OBJECTIVE
Ensure cache entries are keyed and varied correctly, contain only intended data, and are invalidated reliably.

PROJECT CONTEXT
Inspect the existing repository and adapt to its actual framework, database, hosting provider, and conventions. Do
not replace working architecture without evidence.

REQUIRED WORK
1. Inventory CDN, reverse-proxy, framework, browser, API, and Redis caches, including cache-control headers and
surrogate keys.
2. Review every response that varies by Authorization, Cookie, tenant, locale, role, query parameter, or request
header. Define explicit safe cache variation or disable shared caching.
3. Prevent untrusted Host, forwarded headers, query parameters, and user-controlled metadata from influencing cache
keys or generated absolute URLs unsafely.
4. Set appropriate Cache-Control directives for public, private, and sensitive responses. Use no-store for
authentication and other highly sensitive responses where appropriate.
5. Implement explicit invalidation after writes, deletes, permission changes, and account suspension.
6. Ensure cache key serialization is canonical and collision-resistant; include schema/version prefixes for safe
migrations.
7. Add tests that request the same route as two users and two tenants and verify there is no cross-user response
reuse.

SECURITY AND QUALITY RULES
- Treat the server as the trust boundary; never rely on client-side checks for authorization.
- Validate all untrusted input on the server using schemas and allowlists.
- Use least privilege, safe defaults, clear errors, and structured audit logging.
- Do not expose secrets, tokens, personal data, stack traces, or internal identifiers in client responses or logs.
- Preserve existing features and write focused, maintainable changes.

TESTS AND ACCEPTANCE
- Authenticated and tenant-specific responses are never served to the wrong identity.
- Sensitive endpoints have deliberate cache headers.
- Writes invalidate or refresh all relevant cache layers.
- Host-header and header-variation tests do not poison shared cache entries.

IMPLEMENTATION EXPECTATION
First inspect the codebase and explain the relevant current flow briefly. Then implement the changes, not merely
describe them. If a required secret or provider setting is missing, add a clearly named environment-variable
placeholder and document how an authorized maintainer configures it. Never invent credentials.
```

11. Backend resilience, timeouts, retries and circuit breakers

COPY-READY BUILDER PROMPT

```
ROLE
Act as a senior application security engineer, backend architect, and pragmatic code reviewer.

OBJECTIVE
Prevent cascading failures and resource exhaustion while maintaining predictable user-facing behavior.

PROJECT CONTEXT
Inspect the existing repository and adapt to its actual framework, database, hosting provider, and conventions. Do
not replace working architecture without evidence.

REQUIRED WORK
1. Map synchronous and asynchronous dependencies: database, Redis, object storage, email, payment, AI APIs, and
internal services.
2. Set explicit connection, read, write, request, and job timeouts based on measured service expectations.
3. Use bounded retries with exponential backoff and jitter only for transient, idempotent operations. Do not blindly
retry non-idempotent payments or writes.
4. Add circuit breakers and concurrency limits around fragile external dependencies.
5. Use bounded queues with backpressure and dead-letter handling for asynchronous work.
6. Propagate cancellation and request deadlines where supported.
7. Define graceful degradation for non-critical features and safe user messages when a dependency is unavailable.
8. Ensure shutdown drains or safely requeues work and closes connections.

SECURITY AND QUALITY RULES
- Treat the server as the trust boundary; never rely on client-side checks for authorization.
- Validate all untrusted input on the server using schemas and allowlists.
- Use least privilege, safe defaults, clear errors, and structured audit logging.
- Do not expose secrets, tokens, personal data, stack traces, or internal identifiers in client responses or logs.
- Preserve existing features and write focused, maintainable changes.

TESTS AND ACCEPTANCE
- A simulated downstream timeout does not exhaust all application workers.
- Retries are bounded and do not duplicate non-idempotent actions.
- Queue size, retry count, and dead-letter volume are observable.
- Circuit breakers recover after dependency health returns.

IMPLEMENTATION EXPECTATION
First inspect the codebase and explain the relevant current flow briefly. Then implement the changes, not merely
describe them. If a required secret or provider setting is missing, add a clearly named environment-variable
placeholder and document how an authorized maintainer configures it. Never invent credentials.
```

12. Database performance, pooling and query safety

COPY-READY BUILDER PROMPT

```
ROLE
Act as a senior application security engineer, backend architect, and pragmatic code reviewer.

OBJECTIVE
Prevent connection exhaustion, slow queries, unbounded scans, unsafe transactions, and inconsistent concurrent
updates.

PROJECT CONTEXT
Inspect the existing repository and adapt to its actual framework, database, hosting provider, and conventions. Do
not replace working architecture without evidence.

REQUIRED WORK
1. Inspect schema, indexes, ORM usage, connection pool settings, and the highest-cost query paths.
2. Add or adjust indexes based on query plans and observed workload; avoid speculative indexes that add unnecessary
write cost.
3. Use bounded pagination and maximum page sizes. Avoid unbounded list endpoints and N+1 query patterns.
4. Configure connection pools for the deployment model and database limits. Do not create a new database connection
for every request without pooling.
5. Use transactions for related state changes and enforce uniqueness or idempotency at the database layer where
needed.
6. Use read replicas only for workloads that tolerate replication lag; keep writes and read-after-write flows on the
primary when required.
7. Set statement timeouts and monitor slow queries. Ensure user-controlled filters cannot generate unbounded
expensive query plans.

SECURITY AND QUALITY RULES
- Treat the server as the trust boundary; never rely on client-side checks for authorization.
- Validate all untrusted input on the server using schemas and allowlists.
- Use least privilege, safe defaults, clear errors, and structured audit logging.
- Do not expose secrets, tokens, personal data, stack traces, or internal identifiers in client responses or logs.
- Preserve existing features and write focused, maintainable changes.

TESTS AND ACCEPTANCE
- Load tests stay within configured connection limits.
- Critical queries have reviewed query plans and suitable indexes.
- Pagination and filter limits are enforced server-side.
- Concurrent entitlement, billing, and inventory updates remain consistent.

IMPLEMENTATION EXPECTATION
First inspect the codebase and explain the relevant current flow briefly. Then implement the changes, not merely
describe them. If a required secret or provider setting is missing, add a clearly named environment-variable
placeholder and document how an authorized maintainer configures it. Never invent credentials.
```

13. Load balancing, health checks and deployment topology

Covers NGINX/reverse-proxy routing, round robin, weighted routing, least connections, health checks, and container deployment.

COPY-READY BUILDER PROMPT

```
ROLE
Act as a senior application security engineer, backend architect, and pragmatic code reviewer.

OBJECTIVE
Distribute traffic safely across healthy application instances and avoid sending traffic to unhealthy or overloaded
nodes.

PROJECT CONTEXT
Inspect the existing repository and adapt to its actual framework, database, hosting provider, and conventions. Do
not replace working architecture without evidence.

REQUIRED WORK
1. Inspect the actual hosting environment, container setup, reverse proxy, TLS termination, and application port
configuration.
2. Choose a routing strategy appropriate to workload: round robin for similar stateless instances, least connections
for variable request durations, or weighted routing for differing capacity.
3. Configure active health checks and remove unhealthy instances from rotation. Distinguish liveness from readiness
checks.
4. Set safe upstream connection, keepalive, request, and response timeouts. Configure body-size limits consistently
with upload policy.
5. Forward client IP and scheme only through trusted proxy configuration; configure the app’s trusted-proxy list
correctly.
6. Use graceful draining during deploys and autoscaling events. Ensure sessions and critical state are not held only
in process memory.
7. Provide a working container orchestration or Compose configuration only if appropriate for the existing
deployment; do not introduce Docker if the platform does not use it.
8. Document TLS, health-check paths, ports, environment variables, resource requests/limits, and rollback steps.

SECURITY AND QUALITY RULES
- Treat the server as the trust boundary; never rely on client-side checks for authorization.
- Validate all untrusted input on the server using schemas and allowlists.
- Use least privilege, safe defaults, clear errors, and structured audit logging.
- Do not expose secrets, tokens, personal data, stack traces, or internal identifiers in client responses or logs.
- Preserve existing features and write focused, maintainable changes.

TESTS AND ACCEPTANCE
- Unhealthy instances stop receiving new traffic.
- A rolling deployment does not interrupt active requests beyond the documented tolerance.
- The service remains functional when one application instance is removed.
- Proxy headers cannot be spoofed by direct untrusted clients.

IMPLEMENTATION EXPECTATION
First inspect the codebase and explain the relevant current flow briefly. Then implement the changes, not merely
describe them. If a required secret or provider setting is missing, add a clearly named environment-variable
placeholder and document how an authorized maintainer configures it. Never invent credentials.
```

14. DDoS readiness, edge controls and graceful degradation

COPY-READY BUILDER PROMPT

```
ROLE
Act as a senior application security engineer, backend architect, and pragmatic code reviewer.

OBJECTIVE
Improve availability during traffic surges and abusive request patterns without providing attack tooling or weakening
legitimate access.

PROJECT CONTEXT
Inspect the existing repository and adapt to its actual framework, database, hosting provider, and conventions. Do
not replace working architecture without evidence.

REQUIRED WORK
1. Identify public entry points and the most expensive operations. Put managed CDN/WAF or hosting-provider
protections in front of public traffic where available.
2. Apply layered limits: edge request controls, application rate limits, per-account quotas, concurrency bounds, and
queue limits.
3. Protect expensive search, export, upload, AI, and report-generation endpoints with authentication, quotas, bounded
inputs, and asynchronous processing where suitable.
4. Define overload behavior: shed non-critical work, serve safe cached content, pause expensive background tasks, and
preserve core account and billing integrity.
5. Create dashboards and alerts for request volume, error rate, latency, origin saturation, queue depth, and provider
mitigation events.
6. Document provider escalation, emergency configuration changes, status-page messaging, and recovery checks.

SECURITY AND QUALITY RULES
- Use managed defensive controls and authorized load testing only.
- Do not create scripts or instructions for flooding, bypassing, or overwhelming third-party services.
- Do not silently drop critical billing or account operations; preserve idempotency and auditability.

TESTS AND ACCEPTANCE
- Configured quotas and concurrency caps are tested under an authorized staging load test.
- The application degrades gracefully when non-critical dependencies are unavailable.
- Alerts identify saturation before complete service failure.

IMPLEMENTATION EXPECTATION
First inspect the codebase and explain the relevant current flow briefly. Then implement the changes, not merely
describe them. If a required secret or provider setting is missing, add a clearly named environment-variable
placeholder and document how an authorized maintainer configures it. Never invent credentials.
```

15. Frontend security: XSS, CSRF and browser hardening

COPY-READY BUILDER PROMPT

```
ROLE
Act as a senior application security engineer, backend architect, and pragmatic code reviewer.

OBJECTIVE
Reduce XSS, CSRF, clickjacking, unsafe redirects, insecure browser storage, and accidental exposure of sensitive
information.

PROJECT CONTEXT
Inspect the existing repository and adapt to its actual framework, database, hosting provider, and conventions. Do
not replace working architecture without evidence.

REQUIRED WORK
1. Audit dynamic rendering, rich-text editors, Markdown renderers, URL handling, DOM APIs, and any use of
dangerouslySetInnerHTML or equivalent.
2. Prefer framework-safe rendering and context-aware escaping. Sanitize allowed rich text with a maintained allowlist
sanitizer.
3. Set a Content Security Policy appropriate to the app, avoiding unsafe-inline and unsafe-eval where feasible. Roll
out in report-only mode first if necessary.
4. Set security headers including HSTS (when HTTPS is fully deployed), X-Content-Type-Options, Referrer-Policy, and
frame-ancestors or equivalent clickjacking protection.
5. Protect cookie-authenticated state changes against CSRF using framework-supported tokens and origin validation.
6. Validate redirect destinations against an allowlist to prevent open redirects.
7. Keep sensitive tokens out of localStorage and browser logs. Avoid exposing internal API responses or debug state
in production.

SECURITY AND QUALITY RULES
- Treat the server as the trust boundary; never rely on client-side checks for authorization.
- Validate all untrusted input on the server using schemas and allowlists.
- Use least privilege, safe defaults, clear errors, and structured audit logging.
- Do not expose secrets, tokens, personal data, stack traces, or internal identifiers in client responses or logs.
- Preserve existing features and write focused, maintainable changes.

TESTS AND ACCEPTANCE
- Untrusted text is rendered as text, not executable markup.
- State-changing requests reject invalid cross-site requests.
- CSP and security headers are present and do not break required app flows.
- External redirects are restricted to approved destinations.

IMPLEMENTATION EXPECTATION
First inspect the codebase and explain the relevant current flow briefly. Then implement the changes, not merely
describe them. If a required secret or provider setting is missing, add a clearly named environment-variable
placeholder and document how an authorized maintainer configures it. Never invent credentials.
```

16. API security, IDOR and sensitive data exposure

COPY-READY BUILDER PROMPT

```
ROLE
Act as a senior application security engineer, backend architect, and pragmatic code reviewer.

OBJECTIVE
Prevent users from reading or changing records they do not own and prevent APIs from returning unnecessary personal
or internal data.

PROJECT CONTEXT
Inspect the existing repository and adapt to its actual framework, database, hosting provider, and conventions. Do
not replace working architecture without evidence.

REQUIRED WORK
1. Inventory endpoints and identify the data object each endpoint reads or mutates.
2. For every object ID supplied by a client, load the object within the caller’s authorized scope rather than
fetching globally and trusting the client.
3. Use explicit response DTOs or serializers that allowlist fields. Never return database rows wholesale.
4. Check authorization for nested resources, exports, file downloads, search results, and batch operations.
5. Validate pagination, sorting, filters, and field selection against allowlists.
6. Apply request size, response size, and query complexity limits.
7. Review CORS configuration and allow only required origins, methods, and headers. Never combine wildcard origins
with credentialed requests.
8. Ensure API keys are scoped, hashed at rest where appropriate, revocable, and never returned after creation.

SECURITY AND QUALITY RULES
- Treat the server as the trust boundary; never rely on client-side checks for authorization.
- Validate all untrusted input on the server using schemas and allowlists.
- Use least privilege, safe defaults, clear errors, and structured audit logging.
- Do not expose secrets, tokens, personal data, stack traces, or internal identifiers in client responses or logs.
- Preserve existing features and write focused, maintainable changes.

TESTS AND ACCEPTANCE
- Object-ID substitution tests fail safely for unauthorized resources.
- Response schemas contain only intended public fields.
- CORS behavior matches the documented client origins.
- Batch endpoints enforce per-object authorization for every item.

IMPLEMENTATION EXPECTATION
First inspect the codebase and explain the relevant current flow briefly. Then implement the changes, not merely
describe them. If a required secret or provider setting is missing, add a clearly named environment-variable
placeholder and document how an authorized maintainer configures it. Never invent credentials.
```

17. Logging, monitoring, alerting and incident response

COPY-READY BUILDER PROMPT

```
ROLE
Act as a senior application security engineer, backend architect, and pragmatic code reviewer.

OBJECTIVE
Make suspicious activity and production failures detectable without collecting unnecessary sensitive data.

PROJECT CONTEXT
Inspect the existing repository and adapt to its actual framework, database, hosting provider, and conventions. Do
not replace working architecture without evidence.

REQUIRED WORK
1. Use structured logs with request IDs, timestamps, route names, status codes, latency, and safe actor/resource
identifiers.
2. Redact authorization headers, cookies, passwords, reset links, OTPs, payment data, and sensitive personal fields
before logs leave the process.
3. Record security-relevant events such as login failures, MFA changes, permission changes, secret rotation, upload
quarantine, and admin actions.
4. Create metrics and alerts for authentication anomalies, rate-limit spikes, elevated 4xx/5xx rates, unusual export
volume, queue backlog, database saturation, and dependency failures.
5. Define retention, access control, and deletion policies for logs and audit events.
6. Create a concise incident runbook: triage, containment, credential rotation, evidence preservation, customer
communication, recovery, and post-incident review.

SECURITY AND QUALITY RULES
- Treat the server as the trust boundary; never rely on client-side checks for authorization.
- Validate all untrusted input on the server using schemas and allowlists.
- Use least privilege, safe defaults, clear errors, and structured audit logging.
- Do not expose secrets, tokens, personal data, stack traces, or internal identifiers in client responses or logs.
- Preserve existing features and write focused, maintainable changes.

TESTS AND ACCEPTANCE
- A test secret placed in a simulated request never appears in logs.
- Security events can be traced with a request or correlation ID.
- Alerts have clear thresholds, owners, and response steps.
- Log retention and access permissions are documented.

IMPLEMENTATION EXPECTATION
First inspect the codebase and explain the relevant current flow briefly. Then implement the changes, not merely
describe them. If a required secret or provider setting is missing, add a clearly named environment-variable
placeholder and document how an authorized maintainer configures it. Never invent credentials.
```

18. Production QA and security acceptance checklist

Use this checklist before public launch. Mark each item only after a real test or configuration check; do not treat an AI-generated claim as proof.

Area	Verification	Status
Identity	Login, logout, password reset, email verification, MFA, session expiry and revocation behave correctly.	■ Not tested
Authorization	Every protected API checks identity, role, tenant and resource ownership server-side.	■ Not tested
Database	RLS is enabled where required; migrations and policies are tested using realistic roles.	■ Not tested
Input	All external inputs have server-side schemas, length limits and safe parsing.	■ Not tested
Uploads	Type/signature, size, ownership, storage privacy, scan/quarantine and download headers are tested.	■ Not tested
Secrets	No live secrets are committed, logged, or exposed in client bundles; rotation plan exists.	■ Not tested
Dependencies	Lockfile is enforced; vulnerability and secret scans run in CI.	■ Not tested
Abuse	Distributed rate limits, quotas, concurrency limits and safe 429 responses are verified.	■ Not tested
Caching	Tenant-aware keys, TTL jitter, coalescing and invalidation are tested.	■ Not tested
Resilience	Timeouts, bounded retries, circuit breakers and graceful degradation are tested.	■ Not tested
Infrastructure	TLS, proxy trust, health checks, resource limits, backup and rollback plans are verified.	■ Not tested
Observability	Logs are redacted; security alerts and incident runbooks have owners.	■ Not tested
Privacy	Data collection, retention, user notice and deletion behavior are documented.	■ Not tested
Release	Lint, typecheck, unit tests, integration tests and production build pass.	■ Not tested

19. GEO / LLM discovery files and crawler validation

The first screenshot includes an AI recommendation / Generative Engine Optimization section. This is discoverability work, not a security control; it should never override truthful product information.

COPY-READY BUILDER PROMPT

```
ROLE
Act as a senior application security engineer, backend architect, and pragmatic code reviewer.

OBJECTIVE
Help crawlers understand the product using standard public web documents while preserving accuracy, privacy, and
normal indexing controls.

PROJECT CONTEXT
Inspect the existing repository and adapt to its actual framework, database, hosting provider, and conventions. Do
not replace working architecture without evidence.

REQUIRED WORK
1. Create or update a concise /llms.txt describing the product, core features, intended users, official
documentation, support pages, pricing, and important limitations.
2. If useful, create /llms-full.txt containing links or concise summaries of public documentation. Do not include
private, authenticated, unpublished, or user-specific content.
3. Ensure robots.txt clearly expresses crawl permissions and does not accidentally expose private paths as public
content.
4. Generate a valid sitemap.xml containing canonical public URLs with accurate lastmod values where available.
5. Add canonical URLs, descriptive page titles, metadata, structured data only when it accurately represents visible
page content, and useful internal links.
6. Do not fabricate reviews, performance claims, certifications, customer counts, integrations, or AI endorsements.
7. Validate content type, UTF-8 encoding, status codes, redirects, compression, sitemap XML, robots directives, and
crawler access.
8. Keep files synchronized with the actual product and documentation release process.

SECURITY AND QUALITY RULES
- Never place secrets, internal endpoints, private documents, or personal data in crawler-facing files.
- Do not use robots.txt as access control; protect private content with authentication and authorization.
- Do not claim that an LLM will recommend or rank the product because these files exist.

TESTS AND ACCEPTANCE
- All public discovery files return the expected status and content type.
- Sitemap URLs are canonical, public, and return successful responses.
- Robots rules do not accidentally block important public pages or imply security for private paths.
- Every product claim in llms.txt matches the live website.

IMPLEMENTATION EXPECTATION
First inspect the codebase and explain the relevant current flow briefly. Then implement the changes, not merely
describe them. If a required secret or provider setting is missing, add a clearly named environment-variable
placeholder and document how an authorized maintainer configures it. Never invent credentials.
```

20. Master all-in-one security audit and remediation prompt

Use this when you want an AI coding assistant to inspect the whole project. For best results, run focused prompts first for high-risk areas and use this master prompt as the final integration review.

MASTER PROMPT — COPY THE ENTIRE BLOCK

```
ROLE
Act as a principal application security engineer, full-stack architect, and production reliability reviewer.

MISSION
Audit this repository and implement a prioritized, production-ready remediation plan for security, reliability,
privacy, and operational risks. Work only on systems and environments I own or am authorized to assess.

PHASE 1 — REPOSITORY DISCOVERY
1. Identify framework, language, runtime, auth provider, database, storage, cache, deployment platform, CI/CD, and
external services.
2. Map frontend routes, API endpoints, server actions, webhooks, background jobs, data models, and trust boundaries.
3. Identify existing security controls and avoid duplicating or breaking them.
4. Produce a concise architecture and data-flow summary.

PHASE 2 — SECURITY REVIEW
Review:
- Authentication, secure sessions, email verification, MFA, password reset and account recovery.
- Server-side authorization, RBAC, tenant isolation, object ownership and database RLS.
- Input validation, output encoding, SQL/command injection prevention, CSRF, XSS and safe redirects.
- File upload validation, private storage, scanning, size limits and safe download behavior.
- Secret handling, environment variables, credential rotation and CI/CD permissions.
- Dependency health, lockfiles, build scripts and supply-chain risk.
- Rate limiting, quotas, brute-force resistance and abuse monitoring.
- API object-level access, data minimization, CORS and webhook verification.
- Cache stampede, TTL jitter, request coalescing, cache poisoning and tenant-safe invalidation.
- Database pooling, query bounds, timeouts, retries, circuit breakers and queue backpressure.
- Load balancer health checks, trusted proxy configuration, TLS and graceful deployment.
- Logging redaction, alerting, backups, recovery and incident response.

PHASE 3 — RISK REGISTER
For every finding provide: ID, affected file/component, severity (Critical/High/Medium/Low), evidence, impact,
likelihood, recommended fix, dependencies, and test needed. Do not invent vulnerabilities; distinguish confirmed
findings from hypotheses and items that could not be verified.

PHASE 4 — IMPLEMENTATION
1. Fix confirmed Critical and High findings first, then Medium and Low findings.
2. Make small, reviewable changes with clear types and maintainable abstractions.
3. Preserve product behavior and existing data. Provide migrations only when necessary and include rollback guidance.
4. Use deny-by-default authorization and server-side validation.
5. Do not introduce invasive tracking, store raw payment data, expose secrets, or weaken privacy.
6. Do not run destructive production actions. Do not alter live data or deploy without explicit approval.
7. If provider credentials or infrastructure access are missing, document the exact manual configuration needed;
never fabricate credentials.

PHASE 5 — VERIFICATION
1. Add unit, integration, authorization, and regression tests for each fix.
2. Test anonymous, ordinary user, privileged user, and cross-tenant scenarios where relevant.
3. Run formatting, lint, type checking, tests, dependency audit, and production build.
4. Report commands run and their actual results. Never claim a test passed if it was not run.
5. Review changed files for secrets, debug output, unsafe defaults, and accidental data exposure.

FINAL REPORT
Return:
A. Executive summary.
B. Confirmed findings and severity.
C. Changes made, grouped by issue.
D. Tests run with pass/fail and limitations.
E. Required environment variables and deployment settings (names only; never values).
F. Database migration and rollback instructions.
G. Remaining risks and recommended next actions.
H. A launch-readiness checklist.

Begin with repository discovery. Do not guess the stack. Do not stop after writing a plan: implement safe, authorized
fixes and verify them.
```

Recommended execution order

- Start with authentication, authorization, secrets, uploads, and input validation.
- Next address rate limits, database safety, caching, and backend resilience.
- Then harden infrastructure, monitoring, backups, and deployment.
- Finish with the full QA checklist and a fresh independent review.

Appendix A. Secure implementation defaults

Area	Recommended baseline
Sessions	Secure + HttpOnly cookies; SameSite policy chosen for the app; server-side authorization; rotation and revocation.
Passwords	Use a maintained auth provider and modern password hashing; never store plaintext passwords.
Authorization	Deny by default; check ownership and tenant scope on every server-side operation.
Database	Parameterized queries; RLS where supported; least-privilege database roles; bounded pools.
Uploads	Allowlist types; verify signatures; random object keys; private storage; size limits; safe download origin.
Rate limits	Shared/distributed counters; per-route and per-identity policies; bounded retry guidance.
Caching	Tenant-aware keys; TTL jitter; request coalescing; explicit invalidation; no sensitive shared caching.
Secrets	Secret manager; server-only access; rotation after exposure; no secrets in client bundles or logs.
Resilience	Timeouts; bounded retries with jitter; circuit breakers; queue backpressure; graceful degradation.
Monitoring	Structured redacted logs; security events; actionable alerts; tested recovery procedures.

Appendix B. Final instruction to your AI builder

SHORT VERIFICATION PROMPT

```
Review the changes you just made as an independent security reviewer. Do not assume your implementation is correct. Trace each changed flow from the browser to the server, database, cache, and storage provider. Verify authorization at the server boundary, check tenant isolation, inspect logs for sensitive data, review failure behavior, and run the relevant automated tests. Report confirmed defects with evidence and fix them safely. Clearly list anything that requires a human to configure or verify. Do not claim production security or compliance certification.
```

Disclaimer: This guide is a practical engineering aid. Security requirements vary by jurisdiction, provider, data sensitivity, and architecture. Have qualified reviewers assess high-risk systems before launch.